

CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

📖 Project Description

The Emotion Detection & Learning Support Engine is an AI-powered web application designed to analyse students' emotions from text and provide personalized learning support. The system uses **BERT** and BiLSTM deep learning models to detect emotions such as **Confident**, **Curious**, **Confused**, **Frustrated**, and **Bored** with confidence scores. It also supports mixed emotion detection, enabling a more accurate understanding of a student's emotional state. Built using Streamlit, the application offers an intuitive and interactive interface where users can enter their learning concerns, view emotion predictions, and receive instant feedback.

To enhance the learning experience, the application integrates the **Google Gemini API** to generate empathetic, context-aware learning guidance based on the detected emotion and the student's academic query. Prediction history is stored using **SQLite** and **CSV** files for future reference, while an analytics dashboard developed with **Polly** visualizes emotion trends and confidence scores. By combining **Natural Language Processing (NLP)**, **Deep Learning**, **Generative AI**, and **Data Analytics**, the project provides an intelligent educational support system that helps students improve their learning experience through personalized recommendations and emotion-aware assistance.

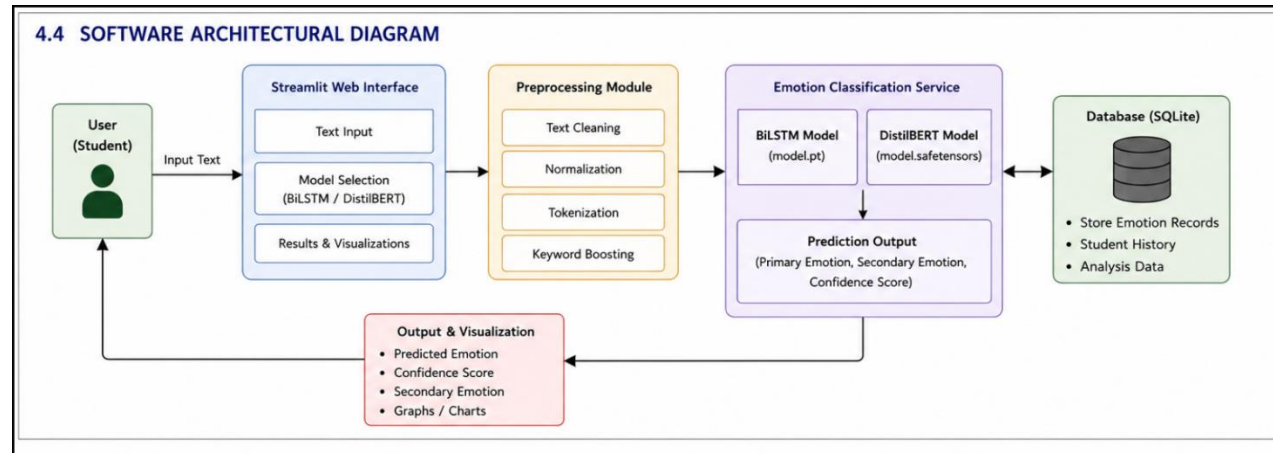
Scenarios

Scenario 1: A user describes a new summer collection of sustainable sunglasses and selects Instagram with a Casual tone. The system generates three emoji-rich captions optimized for Instagram's visual-first audience, ten relevant hashtags mixing broad and niche tags, and three punchy CTAs all in seconds.

Scenario 2: A user promoting a B2B SaaS product selects LinkedIn with a Professional tone. CaptionAI generates thought-leadership-style captions with a value-driven narrative, industry-relevant hashtags, and CTAs designed to drive professional engagement such as demo sign-ups or article reads.

Scenario 3: A user running a weekend flash sale selects Instagram with a Promotional tone. The AI generates urgency-driven captions filled with power words, a curated hashtag set targeting deal-seekers, and CTAs like "Shop before it's gone!" to maximize conversion.

Technical Architecture:



(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites

Description:

The EduEmotion-AI project uses Machine Learning and Deep Learning techniques for detecting students' emotional states from textual input. The application integrates a Streamlit frontend, trained BiLSTM and DistilBERT models, SQLite database, and data preprocessing modules.

Prerequisites:

- **Python Programming:** Python 3.10+ with virtual environment management.
- **Machine Learning Libraries:** PyTorch, Transformers, Scikit-learn, Pandas and NumPy.

- Natural Language Processing: Hugging Face Transformers and text preprocessing techniques.
 - Streamlit Framework: Knowledge of Streamlit for interactive web application development.
 - Git & GitHub: Version control and project deployment.
 - Database: SQLite for storing student emotion records.
 - Model Training: Understanding of BiLSTM and DistilBERT model training and inference.
 - Testing & Deployment: Streamlit Cloud deployment and GitHub integration.
-

Project Workflow

Milestone 1: Dataset Preparation and Model Training

Activity 1.1:

Set up the development environment by creating a virtual environment and installing all dependencies from requirements.txt.

Activity 1.2:

Prepare and preprocess the student emotions dataset by cleaning text, encoding emotion labels, and splitting the dataset into training and testing sets.

Activity 1.3:

Train the custom BiLSTM model and save the trained weights (model.pt) along with the vocabulary (vocab.json).

Activity 1.4:

Fine-tune the DistilBERT model using the processed dataset and export the trained model as model.safetensors with tokenizer files.

Milestone 2: Emotion Classification Pipeline

Activity 2.1:

Develop text preprocessing utilities including text cleaning, normalization and keyword boosting.

Activity 2.2:

Implement the BiLSTM prediction pipeline for emotion classification.

Activity 2.3:

Implement the DistilBERT prediction pipeline for higher accuracy emotion detection.

Activity 2.4:

Generate prediction scores, confidence values and identify primary and secondary emotions.

Milestone 3: Backend Integration

Activity 3.1:

Develop the emotion classifier service responsible for loading trained models during application startup.

Activity 3.2:

Implement database connectivity for storing student emotion records.

Activity 3.3:

Connect preprocessing, prediction and database modules into a unified emotion detection pipeline.

Milestone 4: Streamlit Frontend Development

Activity 4.1:

Design an intuitive Streamlit interface allowing students to enter textual responses.

Activity 4.2:

Display the predicted emotion, confidence score, secondary emotion and graphical analysis.

Activity 4.3:

Provide a responsive interface with model selection between BiLSTM and DistilBERT.

Milestone 5: Testing and Deployment

Activity 5.1:

Verify prediction accuracy using both trained models.

Activity 5.2:

Test model loading, database operations and user interface.

Activity 5.3:

Deploy the application on Streamlit Cloud with trained models stored in the GitHub repository.